Software Requirements Specification

Food Truck Order Management System

Living Document – Agile Approach

Team Sleepers

Hassan Yousef	13006567
Sara Adel	14003723
Hana Yasser	13003628
Mohamed Walid	13006513
Omar Hani	13007515
Abdelhamid ElSharnouby	13006294
Hanin Mohamed	13007010
Khaled Khaled	14001048

German International University (GIU)

Supervised by:

Dr. Iman Awaad
Eng. Amir Haytham

November 21, 2025

Contents

1 Introduction . . . **1**
1.1 Purpose . . . 1
1.2 Intended Audience . . . 1
1.3 Scope . . . 1
1.4 Overview . . . 2

2 Product Vision and Scope . . . **3**
2.1 Product Vision . . . 3
2.2 In Scope . . . 3
2.2.1 Customer Features . . . 3
2.2.2 Vendor Features . . . 3
2.2.3 Admin Features . . . 4
2.3 Out of Scope . . . 4

3 High-Level Architecture . . . **5**
3.1 Password Security: Bcrypt Hashing Implementation . . . 6
3.1.1 Purpose and Security Rationale . . . 6
3.1.2 Why Bcrypt Over Alternative Approaches . . . 6
3.1.3 Technical Implementation . . . 6
3.1.4 Security Configuration . . . 6
3.1.5 Integration Points . . . 6
3.1.6 Security Benefits and Threat Mitigation . . . 6
3.1.7 Performance Considerations . . . 6
3.1.8 Compliance and Best Practices . . . 6
3.1.9 Future Enhancements . . . 6
3.2 Data Preservation Strategy: Soft Deletes . . . 8
3.2.1 Purpose and Rationale . . . 8
3.2.2 Implementation Architecture . . . 8
3.2.3 Application Layer Logic . . . 8
3.2.4 System Impact and Considerations . . . 8
3.2.5 User Experience Impact . . . 8
3.2.6 Data Lifecycle Management . . . 8

4 Functional Requirements . . . **9**
4.1 User Stories . . . 9
4.1.1 Food Truck Customer User Stories . . . 9
4.1.2 Food Truck Service Provider User Stories . . . 9
4.1.3 Food Truck Admin User Stories . . . 9
4.2 Use Case Diagrams . . . 10

5 Non-Functional Requirements . . . **13**

6 Interface Requirements . . . **14**

7 Constraints . . . **15**
7.1 Technical Constraints . . . 15
7.2 Operational Constraints . . . 15
7.3 Resource Constraints . . . 15
7.4 Regulatory and Policy Constraints . . . 16

7.5 User and Usability Constraints . . . 16
7.6 Design and Architectural Constraints . . . 16

8 Continuous Integration and Continuous Deployment Pipeline . . . 17
8.1 Overview and Purpose . . . 17
8.2 Pipeline Architecture . . . 17
8.2.1 Technology Stack . . . 17
8.2.2 Pipeline Stages . . . 18
8.3 Branching Strategy and Workflow . . . 20
8.3.1 Git Branching Model . . . 20
8.3.2 Pull Request Workflow . . . 20
8.4 Environment Management . . . 21
8.4.1 Environment Tiers . . . 21
8.4.2 Configuration Management . . . 21
8.5 Automated Testing Strategy . . . 21
8.5.1 Test Coverage Requirements . . . 21
8.5.2 Test Types and Execution . . . 21
8.6 Deployment Strategies . . . 22
8.6.1 Rolling Deployment . . . 22
8.6.2 Database Migration Handling . . . 22
8.7 Monitoring and Observability . . . 23
8.7.1 Pipeline Monitoring . . . 23
8.7.2 Application Monitoring Post-Deployment . . . 23
8.8 Security and Compliance . . . 23
8.8.1 Pipeline Security . . . 23
8.8.2 Access Controls . . . 23
8.9 Benefits and Impact . . . 24

9 Future Scope of the Project . . . 25

Notes . . . 26

1 Introduction

1.1 Purpose

Problem Statement:

Students and staff at GIU face significant challenges during peak hours when ordering food from campus food trucks. Long waiting lines, unclear preparation times, and lack of order visibility create frustration and time loss, especially between classes when time is limited.

Solution:

The GIU Food Truck Order Management System is a comprehensive web-based application designed to enable the members of the GIU community to reserve menu items from specific food trucks on campus for pick-up within a given time window, thus alleviating the problem with long waiting times for food and beverage items during peak periods in between classes.

This system enables seamless menu browsing, order placement, and order management for food truck customers as well as providing a similarly intuitive, efficient, and scalable solution for food service providers.

1.2 Intended Audience

- **Project Team:** Hassan Yousef, Sara Adel, Hana Yasser, Mohamed Walid, Omar Hany, Abdelhamid ElSharnouby, Hanin Mohamed, Khaled Khaled.
- **Developers and Testers:** Project team members.
- **Product Owner:** Doctor Iman Awaad.
- **Instructors:** Dr. Iman Awaad, Eng. Amir Haytham and Eng. Ahmad Sherif

1.3 Scope

The Food-truck System is a website that helps students and staff at GIU order food from the food trucks on campus. People can open the menus of different food trucks to see what food and drinks are available. They can then choose the items they want and make their order online. The system lets them pick up their food in a specific time, so they don't have to wait in long lines. This makes it easier for everyone to get their food, especially during busy times between classes. The system also helps the food trucks manage orders better, making sure everything runs smoothly for both customers and service providers. In general, the goal is to save time and make ordering food on campus simple and comfortable for the entire community.

1.4 Overview

The Food Truck Order Management System is a simple website for ordering food from the food trucks on the GIU campus. It helps to reduce waiting in lines by letting users place orders before going to the truck.

Who uses it:

- **Students and Staff (Customers):** Browse trucks, view menus, select items, place an order, choose (or accept) a pickup time, and check the order status (received, preparing, ready) on the website.
- **Food Truck Staff:** See incoming orders, update each order's status, manage menu items (add or remove).
- **Administrator:** Add or remove food trucks and oversee basic platform usage.

How order tracking works: The page refreshes (or auto-checks) every short interval to show updated order status. It does not use push notifications or advanced real-time technology.

What it does NOT include: No delivery, no online payment, no mobile app, no live chat, no push notifications, no multi-language (English only), no external integrations.

Why provider hours are longer: Food truck staff have extra time (before and after customer hours) to prepare ingredients and clean up.

Main goal: Save time, make ordering clearer and easier, and help trucks manage orders better.

Other notes: Works in a normal web browser on phones, tablets, and laptops. Designed for up to about 500 users at once.

2 Product Vision and Scope

2.1 Product Vision

The **Food Truck Order Management System** envisions a seamless, digital-first campus dining experience that minimizes waiting time and optimizes operations for both customers and vendors. The system will serve as a centralized web platform connecting GIU students, staff, and food truck operators through a polling-based ordering process.

From a user perspective, the vision is to eliminate the inefficiency of physical queues by allowing users to browse menus, place customized orders, and schedule pickups in advance. For food truck operators, the platform provides a unified dashboard to manage incoming orders and track preparation progress. Administrators will be able to manage participating vendors, monitor overall platform activity, and ensure service quality across all trucks.

The long-term vision is to transform how on-campus food services operate — creating a scalable digital ecosystem that supports automated order tracking, predictive demand analytics, and an integrated payment experience. Ultimately, this system aims to enhance convenience, transparency, and satisfaction for the GIU community while providing actionable insights to vendors and administrators for continuous operational improvement.

2.2 In Scope

The system includes the following features as part of its core functionality, categorized by user role.

2.2.1 Customer Features

- **User Authentication:** Customers can register and log in using email and password.
- **Menu Browsing:** Users can browse a list of food trucks, then view each truck's menu of food items.
- **Online Ordering:** Customers can place orders and select pickup times.
- **Estimated Preparation Time:** The system displays estimated time before confirming an order.
- **Order Status Tracking:** Customers can track order progress (Pending, In Progress, Ready).
- **Busy Mode on menu:** Customers can see if a vendor is currently too busy to accept new orders.

2.2.2 Vendor Features

- **View and Manage Orders:** Vendors can view incoming orders and update their statuses.

- **Menu Management:** Vendors can add or remove menu items.
- **Activate Busy Mode:** Vendors can temporarily stop accepting new orders.

2.2.3 Admin Features

- **Food truck management:** Admin can add or remove food trucks.

2.3 Out of Scope

The following features are explicitly excluded from the current system scope:

- **Restaurant Rating:** Users cannot rate or review food trucks or individual menu items. No feedback mechanism or star rating system is included in this release.
- **Bestsellers Menu:** The system does not track or display popular items, trending dishes, or "most ordered" recommendations. No sales analytics or item popularity features are provided.
- **Mobile Application:** The system will only be accessible via web browsers; no Android or iOS apps are included in this release.
- **Online Payment Integration:** Users will not be able to pay online. All transactions will be handled offline or through existing university methods.
- **Delivery Services:** Food delivery is not supported. Customers must pick up their orders from the food trucks in person.
- **University Authentication System Integration:** The system does not connect to GIU's single sign-on or internal authentication infrastructure.
- **Push Notifications:** Real-time push notifications (e.g., mobile or browser alerts) are not supported in this release. Users must track their orders through the website interface.
- **Multiple Language Support:** The system will only support English in its initial release. Support for other languages is excluded from scope.
- **Live Chat or Messaging:** No communication feature (e.g., chat with vendors or support) is included.
- **Advanced Real-Time Communication:** The system will use simple periodic refresh or polling to check order updates. Advanced real-time communication technologies (WebSockets, Server-Sent Events) are not included in this version.
- **Third-Party Integrations:** No integration with external services (e.g., social media, third-party APIs) is planned.
- **Advanced Analytics:** The system will not include complex analytics or reporting features for vendors or administrators.

3 High-Level Architecture

- **Front End:** Modern HTML, CSS, JavaScript, and jQuery. Or JavaScript Framework
- **Back End:** RESTful API using Node.js
- **Database:** Relational (PostgreSQL)
- **Authentication:** Email/password authentication (no external OAuth)
- **Payment:** Currently, none (offline transactions only)
- **Hosting:** To be determined (university servers or cloud provider)

3.1 Password Security: Bcrypt Hashing Implementation

3.1.1 Purpose and Security Rationale

The Food Truck Order Management System implements **bcrypt** as the cryptographic hashing algorithm for all user password storage. This industry-standard approach ensures that user credentials remain secure even in the event of database compromise, protecting students, staff, and vendors from unauthorized access.

Core Security Principles:

- **Irreversibility:** Bcrypt is a one-way cryptographic function. Passwords cannot be reverse-engineered from stored hashes, ensuring that even system administrators cannot view user passwords.
- **Defense Against Rainbow Tables:** Built-in salt generation prevents attackers from using precomputed hash tables to crack passwords efficiently.
- **Computational Cost:** Bcrypt's adaptive work factor makes brute-force attacks computationally expensive and time-prohibitive.
- **Future-Proof Security:** The configurable cost factor allows increasing computational complexity as hardware capabilities improve.

3.1.2 Why Bcrypt Over Alternative Approaches

Comparison with Other Hashing Methods:

MD5 and SHA-1 (Rejected):
These algorithms are cryptographically broken and vulnerable to collision attacks. They hash passwords too quickly, enabling millions of guesses per second with modern GPUs.

Plain SHA-256/SHA-512 (Insufficient):
While cryptographically secure for integrity checking, these algorithms are too fast for password hashing. Without proper salting and key stretching, they remain vulnerable to brute-force attacks.

PBKDF2 (Alternative):
A viable alternative that provides key stretching, but bcrypt is specifically designed for password hashing with built-in salt management and has stronger resistance to GPU-based attacks.

Argon2 (Future Consideration):
The winner of the Password Hashing Competition (2015), Argon2 offers superior protection against GPU and ASIC attacks. However, bcrypt remains the industry standard with broader ecosystem support and proven security track record for web applications.

Scrypt (Alternative):
Provides memory-hard properties that resist hardware-based attacks, but bcrypt's maturity and widespread library support make it the preferred choice for this project's scope.

Decision Rationale: Bcrypt strikes the optimal balance between proven security, performance, ecosystem maturity, and ease of implementation for a university-scale web application.

3.1.3 Technical Implementation

Algorithm Mechanics:

3.2 Data Preservation Strategy: Soft Deletes

3.2.1 Purpose and Rationale

The Food Truck Order Management System implements a **soft delete** mechanism as a core data preservation strategy to ensure business continuity, regulatory compliance, and operational integrity. Rather than permanently removing records from the database, soft deletes mark data as deleted while maintaining its physical presence in the system.

Key Objectives:

- **Data Recovery:** Enable restoration of accidentally deleted records without requiring database backups.
- **Audit Trail:** Maintain complete historical records for compliance, reporting, and analysis purposes.
- **Referential Integrity:** Preserve relationships between orders, menu items, and users even after logical deletion.
- **Business Intelligence:** Retain historical data for trend analysis, seasonal patterns, and vendor performance metrics.
- **Legal Compliance:** Support potential retention policies and dispute resolution requirements.

3.2.2 Implementation Architecture

Database Schema Design:
Soft deletes are implemented through the addition of metadata columns to relevant database tables:

- **deleted_at (TIMESTAMP):** Records the exact date and time when a record was logically deleted. NULL indicates an active record.
- **deleted_by (INTEGER/VARCHAR):** Foreign key or identifier referencing the user who performed the deletion (admin, vendor, or system).
- **is_deleted (BOOLEAN):** Optional flag for faster query indexing, typically derived from deleted_at IS NOT NULL.

Affected Entities:
The soft delete mechanism applies to the following core system entities:

- **Menu Items:** When vendors remove items from their menu, records remain in the database to preserve order history.
- **Food Trucks:** Deactivated or removed food trucks retain their data for historical order analysis and reporting.
- **User Accounts:** Deleted user accounts maintain order history and transaction records.
- **Orders:** While orders are rarely deleted, the system supports soft deletion for cancellations requiring audit trails.

3.2.3 Application Layer Logic

Query Filtering:
All standard application queries automatically exclude soft-deleted records through a consistent WHERE clause pattern:

```
SELECT * FROM menu_items
```

4 Functional Requirements

4.1 User Stories

4.1.1 Food Truck Customer User Stories

- As a customer, I want to schedule my lunch order in advance so that I can check the order status through the website and know when my order is ready for pickup.
- As a customer, I want to browse all the food trucks available on campus, see all options.

4.1.2 Food Truck Service Provider User Stories

- As a food truck staff member, I want to receive new orders through the system and update order status (received, preparing, ready) so customers can track their orders via polling.
- As a food truck owner, I want to create my menu with items, prices, and cooking times.

4.1.3 Food Truck Admin User Stories

- As an administrator, I add or remove food trucks.

4.2 Use Case Diagrams

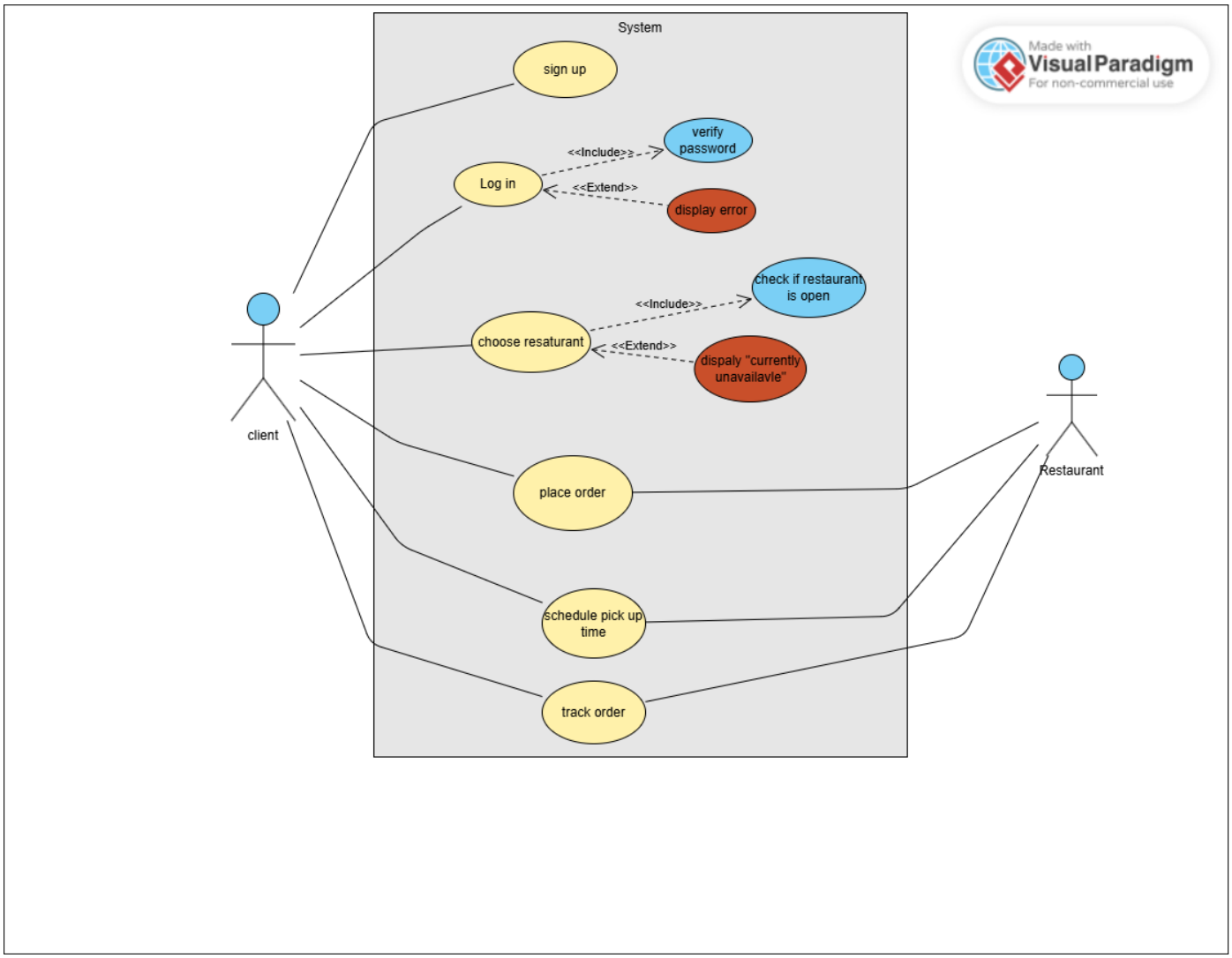

Figure 1: **Client Use Case Diagram**

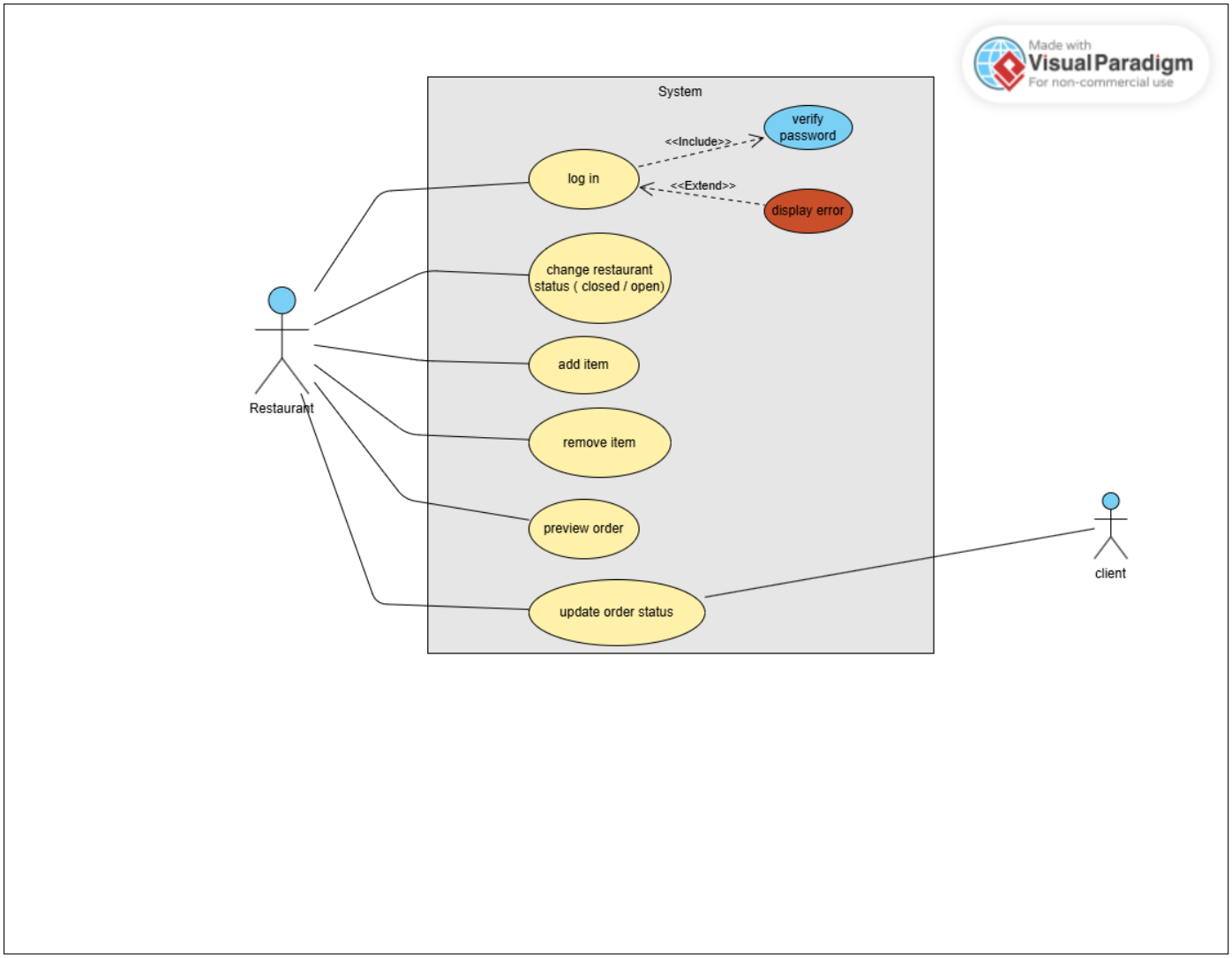

Figure 2: **Restaurant Use Case Diagram**

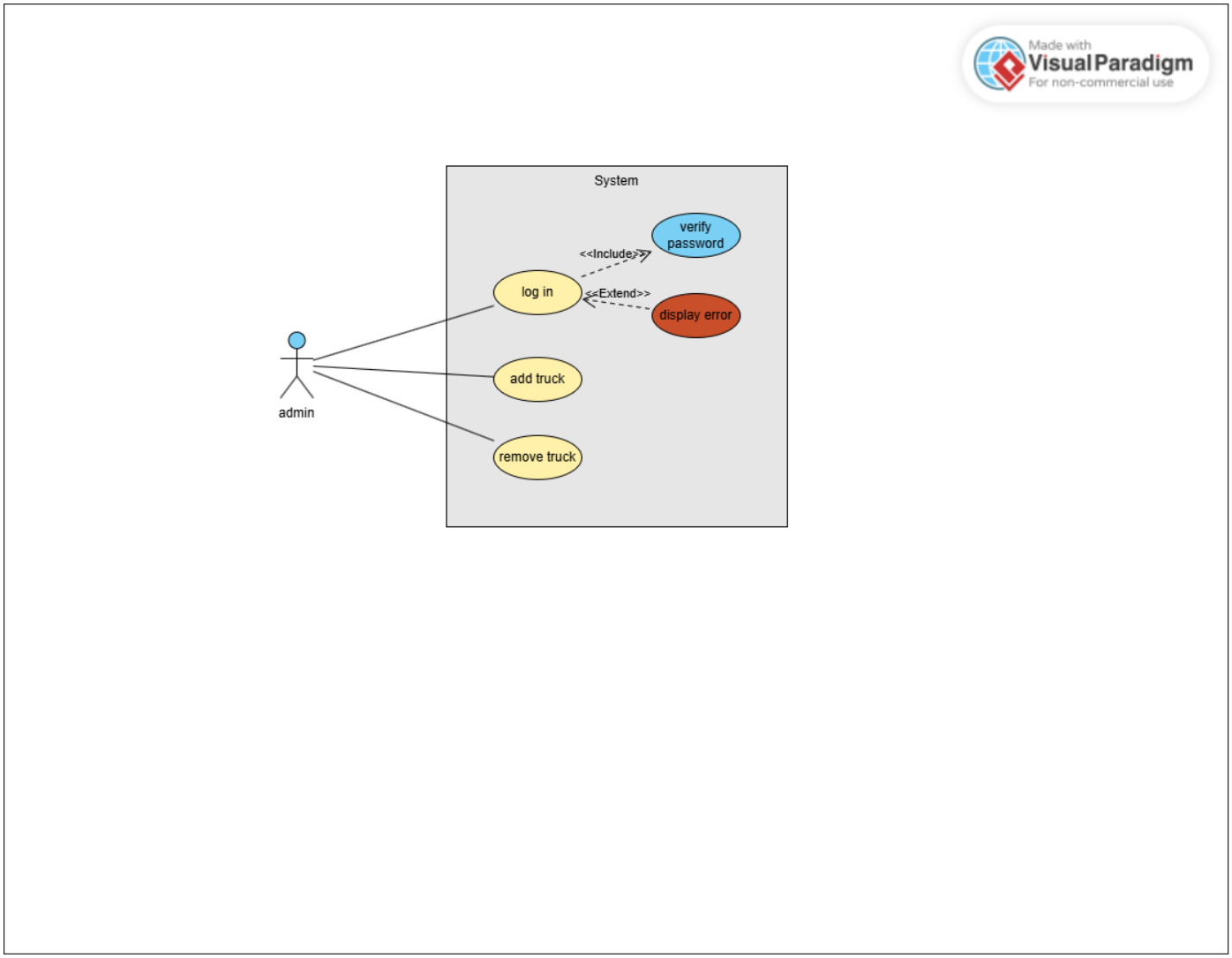

Figure 3: **Admin Use Case Diagram**

5 Non-Functional Requirements

Response Time:
Page load times under three seconds.

Concurrent Users:
Support for at least 500 simultaneous users.

API Security:
Rate limiting and input validation.

Backup & Recovery:
Automated daily backups with disaster recovery plan.

System Operating Hours (Food Service Providers):
7:00 AM - 5:00 PM, Saturday to Thursday. This allows providers to prepare before customer ordering begins and complete cleanup after customer hours end.

System Operating Hours (Food-truck Patrons):
8:00 AM - 4:40 PM, Saturday to Thursday.

6 Interface Requirements

Currently, no external integrations (e.g. credit card, external authentication).

7 Constraints

7.1 Technical Constraints

- **Web-Based Platform:** The system must be accessible via standard web browsers (Chrome, Firefox, Safari, Edge) without requiring native mobile applications in the initial release.

- **Technology Stack:** Development is limited to technologies and frameworks approved by GIU's IT infrastructure and familiar to the development team.

- **Database Management:** Must utilize a relational database system compatible with university hosting environments.

- **No External Payment Integration:** The initial version will not integrate third-party payment gateways; transactions will be handled offline or through existing university payment systems.

- **Campus Network Dependency:** Optimal performance is guaranteed only within GIU campus network infrastructure.

7.2 Operational Constraints

- **Operating Hours:** System functionality is limited to food truck operational hours (7:00 AM - 5:00 PM for providers, 8:00 AM - 4:40 PM for customers, Saturday to Thursday). The extended provider hours allow for preparation before service and cleanup after service.

- **Physical Pickup Required:** Orders must be physically collected from food trucks; no delivery service is provided.

- **Campus-Only Service:** The system serves only GIU campus food trucks and is geographically restricted to on-campus operations.

- **Manual Order Fulfillment:** Food preparation and handoff processes remain manual; the system only manages digital ordering and tracking.

7.3 Resource Constraints

- **Development Timeline:** The project must be completed within the academic semester timeframe as defined by the course requirements.

- **Team Size:** Development is limited to an 8-member student team with varying levels of experience.

- **Budget Limitations:** No dedicated budget for premium third-party services, APIs, or cloud infrastructure beyond what GIU provides.

- **Hardware Infrastructure:** Dependent on existing GIU server infrastructure and food truck operator devices (smartphones, tablets, or PCs).

7.4 Regulatory and Policy Constraints

- **Data Privacy Compliance:** Must adhere to GIU's data protection policies and student information privacy guidelines.
- **University Authentication:** Initial version will not integrate with GIU's central authentication system (planned for future releases).
- **Food Safety Standards:** The system does not enforce or monitor food safety regulations; responsibility remains with food truck operators.
- **Vendor Agreements:** Participation of food trucks is subject to approval and agreements with GIU administration.

7.5 User and Usability Constraints

- **Internet Connectivity:** Users must have stable internet connection to access the system; no offline functionality is provided.
- **Device Requirements:** Users need devices with modern web browsers; minimum screen size recommendations apply for optimal experience.
- **Digital Literacy:** Assumes basic computer and smartphone literacy among students, staff, and food truck operators.
- **Language Support:** Initial release supports English only; Arabic language support planned for future versions.

7.6 Design and Architectural Constraints

- **Scalability Scope:** System designed for GIU campus scale (approximately 500 concurrent users); not architected for multi-campus deployment in initial version.
- **Polling-Based Updates:** Order status updates operate on polling or periodic refresh mechanisms, not true real-time WebSocket connections.
- **Legacy System Integration:** No integration with existing GIU administrative systems (student information, financial systems) in the initial release.
- **Responsive Design Mandatory:** Must function across desktop, tablet, and mobile screen sizes despite being web-only.

8 Continuous Integration and Continuous Deployment Pipeline

8.1 Overview and Purpose

The Food Truck Order Management System implements a robust **CI/CD (Continuous Integration/Continuous Deployment)** pipeline to automate the software development lifecycle, ensuring rapid, reliable, and repeatable deployments. This automated workflow enables the development team to deliver features, bug fixes, and security updates with confidence while maintaining system stability and minimizing manual intervention.

Key Objectives:

- **Automated Testing:** Execute comprehensive test suites on every code change to catch bugs early in the development cycle.
- **Code Quality Assurance:** Enforce coding standards, security best practices, and maintainability through automated analysis.
- **Rapid Feedback:** Provide developers with immediate feedback on code quality and test results within minutes of pushing changes.
- **Deployment Automation:** Eliminate manual deployment steps, reducing human error and accelerating release cycles.
- **Environment Consistency:** Ensure identical deployment processes across development, staging, and production environments.
- **Rollback Capability:** Enable quick reversion to previous stable versions if issues are detected post-deployment.

8.2 Pipeline Architecture

8.2.1 Technology Stack

Version Control:
Git with GitHub as the central repository hosting platform.

CI/CD Platform:
GitHub Actions for pipeline orchestration and automation workflows.

Containerization:
Docker for creating consistent, portable application environments across all deployment stages.

Testing Frameworks:

- Jest for JavaScript unit and integration testing
- Supertest for API endpoint testing
- Playwright or Cypress for end-to-end browser testing

Code Quality Tools:
- ESLint for JavaScript code linting and style enforcement
- Prettier for consistent code formatting
- SonarQube or CodeClimate for code quality metrics and technical debt analysis

Security Scanning:
- npm audit for dependency vulnerability scanning
- Snyk or Dependabot for automated security patch suggestions
- OWASP Dependency-Check for known vulnerability detection

8.2.2 Pipeline Stages

The CI/CD pipeline consists of sequential stages that must pass before code reaches production:

Stage 1: Code Commit and Trigger

- Developer commits code to feature branch and pushes to GitHub.
- GitHub webhook triggers CI/CD pipeline automatically.
- Pipeline executes on pull request creation, update, or merge to main branches.

Stage 2: Build and Environment Setup

- Checkout source code from repository.
- Set up Node.js environment (v18 or v20 LTS).
- Install project dependencies using `npm ci` for reproducible builds.
- Compile TypeScript (if applicable) and bundle frontend assets.
- Build Docker images for backend and frontend services.

Stage 3: Static Code Analysis

- Run ESLint to detect code quality issues, unused variables, and anti-patterns.
- Execute Prettier to verify code formatting consistency.
- Perform SonarQube analysis for code complexity, duplication, and maintainability metrics.
- Generate code coverage reports to ensure adequate test coverage (minimum 70% threshold).

Stage 4: Automated Testing

- **Unit Tests:** Execute isolated tests for individual functions and components (target: 80% coverage).
- **Integration Tests:** Verify interactions between modules, database operations, and API endpoints.
- **API Tests:** Validate RESTful API contracts, response formats, and error handling.
- **End-to-End Tests:** Simulate complete user workflows in headless browser environments.
- All tests must pass with zero failures before proceeding.

Stage 5: Security Scanning

- Run `npm audit` to identify vulnerable dependencies.
- Execute OWASP Dependency-Check for known CVE (Common Vulnerabilities and Exposures).
- Scan Docker images for security vulnerabilities using Trivy or Clair.
- Block deployment if critical or high-severity vulnerabilities detected.

Stage 6: Staging Deployment

- Deploy application to staging environment (identical to production configuration).
- Execute smoke tests to verify critical functionality (login, order placement, status updates).
- Run performance tests to ensure response times meet non-functional requirements.

- Manual review and approval required before production deployment (for critical releases).

Stage 7: Production Deployment

8.3 Branching Strategy and Workflow

8.3.1 Git Branching Model

The project employs a modified **Git Flow** branching strategy optimized for continuous deployment:

main Branch:
Production-ready code. All commits to main trigger automatic production deployment after passing all pipeline stages. Protected branch requiring pull request reviews.

develop Branch:
Integration branch for ongoing development. Merges from feature branches undergo full CI testing. Automatically deploys to staging environment.

feature/* Branches:
Individual feature development. Naming convention: `feature/description-of-feature`. Triggers CI pipeline on push and pull request. Deleted after merge to develop.

bugfix/* Branches:
Bug fixes during development cycle. Similar workflow to feature branches. Naming: `bugfix/issue-description`.

hotfix/* Branches:
Emergency production fixes. Branched directly from main, merged to both main and develop. Triggers expedited pipeline with production deployment.

release/* Branches:
Release preparation and stabilization. Branched from develop when features are complete. Undergo final testing before merging to main.

8.3.2 Pull Request Workflow

1. Developer creates feature branch from develop.
2. Implements feature with regular commits and pushes to remote.
3. Creates pull request targeting develop branch.
4. Automated CI pipeline executes (build, test, lint, security scan).
5. Code review required from at least one team member.
6. All CI checks must pass (green status) before merge allowed.
7. Conflicts resolved, pull request approved and merged.
8. Feature branch automatically deleted, develop branch deploys to staging.

8.4 Environment Management

8.4.1 Environment Tiers

Development Environment:
Local developer machines with Docker Compose for running full stack (Node.js backend, PostgreSQL database, frontend). Environment variables stored in `.env.development` (git-ignored).

Staging Environment:
Cloud-hosted environment mirroring production configuration. Automatically updated on develop branch merges. Uses separate PostgreSQL database with production-like data volume. Accessible only within GIU network or via VPN.

Production Environment:
Live system serving GIU campus users. Deployed only from main branch after manual approval. High availability configuration with load balancing. Strict access controls and monitoring.

8.4.2 Configuration Management

- Environment-specific variables (database credentials, API keys) stored in GitHub Secrets.
- Configuration injected at deployment time, never committed to repository.
- Separate secrets for staging and production environments.
- Secret rotation performed quarterly or immediately after team member departure.

8.5 Automated Testing Strategy

8.5.1 Test Coverage Requirements

- **Overall Code Coverage:** Minimum 70% line coverage across entire codebase.
- **Critical Path Coverage:** 100% coverage for authentication, order processing, and payment-related code.
- **API Endpoint Coverage:** All RESTful endpoints must have corresponding integration tests.
- **Regression Test Suite:** Automated tests for all previously discovered bugs to prevent reoccurrence.

8.5.2 Test Types and Execution

Unit Tests:
Test individual functions, components, and utility methods in isolation. Executed on every commit (fastest feedback, typically completes in under 30 seconds).

Integration Tests:
Verify interactions between modules, database queries, and external service calls. Use test database seeded with consistent fixtures. Execute on pull requests and develop/main merges.

API Tests:
Validate HTTP endpoints, request/response formats, status codes, and error handling. Test authentication flows, authorization rules, and rate limiting. Run on staging environment after deployment.

End-to-End Tests:
Simulate complete user workflows (registration, login, browse menu, place order, track status). Execute in headless browser against staging environment. Run nightly and before production deployments.

Performance Tests:
Load testing with simulated concurrent users (target: 500 simultaneous users). Stress testing to identify breaking points and resource constraints. Execute weekly on staging environment.

8.6 Deployment Strategies

8.6.1 Rolling Deployment

Production deployments use a **rolling update** strategy to minimize downtime:

1. New application version deployed to subset of servers (e.g., 25% of instances).
2. Health checks verify successful startup and functionality.
3. If healthy, deployment continues to next batch of servers.
4. Process repeats until all servers updated.
5. Zero downtime achieved as old instances remain active until new ones ready.
6. Automatic rollback if health checks fail at any stage.

8.6.2 Database Migration Handling

- Schema changes managed through versioned migration scripts (using tool like Flyway or db-migrate).
- Migrations execute automatically before application deployment.
- Backward-compatible migrations ensure old application versions continue functioning during rolling updates.
- Database backups created automatically before migration execution.

- Migration failures trigger automatic rollback and deployment abortion.

8.7 Monitoring and Observability

8.7.1 Pipeline Monitoring

- GitHub Actions dashboard displays pipeline status, execution time, and failure patterns.
- Slack/email notifications sent on pipeline failures with error details and logs.
- Weekly pipeline metrics reported: success rate, average execution time, flaky test identification.
- Failed builds block merging and trigger immediate team notification.

8.7.2 Application Monitoring Post-Deployment

- Application performance monitoring (APM) tracks response times, error rates, and resource utilization.
- Log aggregation centralizes application logs from all servers for analysis and troubleshooting.
- Automated alerts triggered on error rate spikes, performance degradation, or service unavailability.
- Dashboards display key metrics: active users, order volume, API response times, database query performance.

8.8 Security and Compliance

8.8.1 Pipeline Security

- GitHub Actions workflows execute in isolated containers with minimal permissions.
- Secrets encrypted at rest and decrypted only during pipeline execution.
- Dependency updates automatically tested and reviewed before merging (Dependabot integration).
- Audit logs maintained for all deployments, including who approved and when.

8.8.2 Access Controls

- Production deployments require manual approval from designated team leads.
- GitHub branch protection rules enforce code review requirements and status checks.
- SSH keys and cloud credentials rotated regularly and stored securely.

- Principle of least privilege applied to all service accounts and deployment permissions.

8.9 Benefits and Impact

Development Velocity:

- Reduced deployment time from hours to minutes through automation.
- Faster feedback loops enable rapid iteration and bug fixing.
- Developers focus on coding rather than manual testing and deployment tasks.

Quality Assurance:

- Automated testing catches bugs before they reach production.
- Consistent code quality through enforced linting and formatting standards.
- Security vulnerabilities identified and addressed early in development cycle.

Reliability and Stability:

- Repeatable, automated deployments eliminate human error.
- Quick rollback capability minimizes impact of defects.
- Staging environment catches issues before production exposure.

Team Collaboration:

- Transparent pipeline status visible to entire team.
- Code review process integrated into CI/CD workflow.
- Shared responsibility for code quality and deployment success.

9 Future Scope of the Project

Potential features for future versions:

- Integration with university authentication.
- Push notifications for order updates.
- Mobile app version.
- Payment gateway support.
- **Rating and Review System:** Allow customers to rate food trucks and menu items, providing feedback and helping others make informed choices.
- **Inventory Management:** Enable vendors to track stock levels, receive low-stock alerts, and automatically mark items as unavailable when ingredients run out.

Notes

Important Note

- This is a **living document**. It evolves with project development.
- At the end, this document may be retitled as "Project Documentation."

How the System Solves the Problem

The Challenge: Students and staff waste valuable time standing in long queues at food trucks during peak hours, with no visibility into wait times or order status.

Our Solution:

1. **Pre-ordering eliminates queues:** Users browse menus and place orders online before arriving at the truck, eliminating the need to wait in physical lines.
2. **Time visibility reduces uncertainty:** The system displays estimated preparation times and allows users to select pickup windows, helping them plan their schedule.
3. **Order tracking provides transparency:** Customers can check their order status (Pending → In Progress → Ready) through polling-based updates on the website, knowing exactly when to pick up their food.
4. **Busy mode prevents overload:** Vendors can activate busy mode when overwhelmed, preventing new orders and managing capacity effectively.
5. **Centralized platform streamlines operations:** Both customers and vendors access a single unified system, reducing confusion and improving coordination.

Result: Students and staff save time, avoid frustration, and enjoy a smoother campus dining experience, while food truck operators manage orders more efficiently and serve customers better.